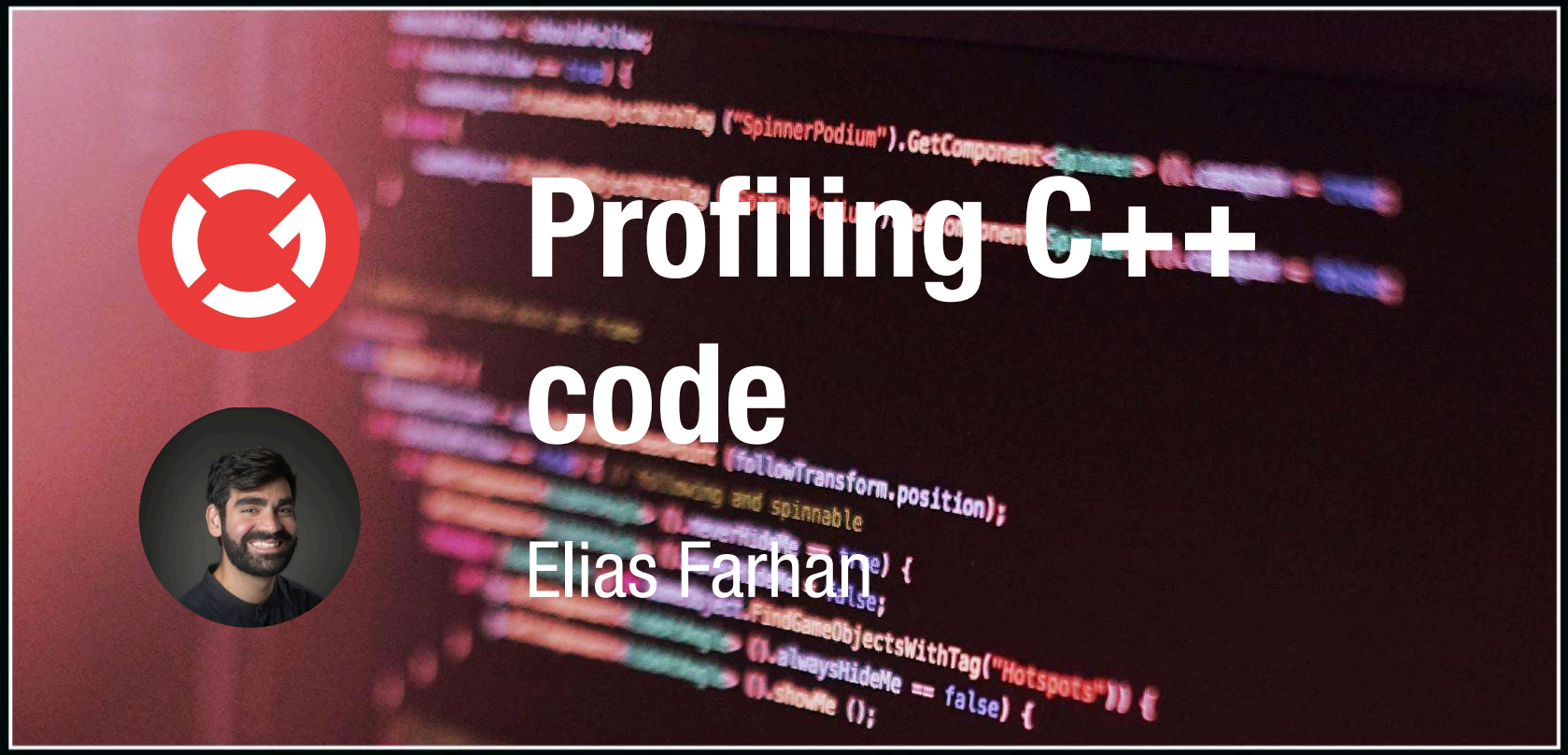
SAE INSTITUTE



Profiling C++ code

Elias Farhan

University of Hertfordshire UH

What are we doing today?

- Measure performance with the right **units**
- Understand why reading code is not enough
- Use **Big-O notation** to reason about how work scales
- Compare **sampling** and **instrumentation** profilers
- Follow a practical profiling loop: find, validate, root-cause, fix
- Separate **CPU time** from **wait time**
- Write code that is safer for **real-time** hot paths
- Choose containers and allocation strategies based on access patterns
- Optimize the code that matters, without over-engineering

Measuring performance

Units of measurement

Use real units when discussing performance. Frame rate is a target or symptom, not a profiling unit.

- Time: seconds, milliseconds, microseconds, nanoseconds
- Memory: bytes, KiB, MiB, GiB
- Information: bits and bytes
- Power: watts
- **Not FPS**

Frame time, not FPS

FPS hides the cost of each frame. Going from 120 FPS to 60 FPS costs about **8.3 ms**. Going from 60 FPS to 30 FPS costs about **16.7 ms**.

FPS	Frame time
120	8.33 ms
60	16.67 ms
30	33.33 ms
20	50.00 ms

Why profiling?

- Figuring out why a program is slow is hard
- Reading the code can easily mislead you
- Modern CPUs are complex
- Algorithmic complexity tells you what might scale badly
- Memory access often matters more than instruction count
- Always measure before optimizing

Algorithmic thinking

Why algorithmic complexity?

Profiling tells where time goes. Complexity helps predict what will explode when the number of entities, tiles, jobs, or resources grows.

- 100 entities can hide bad choices
- 10,000 entities makes scaling visible
- Big-O describes how work grows with input size
- It ignores constants, memory layout, and hardware effects
- It is a reasoning tool, not a benchmark

SAE INSTITUTE

What does n mean?

In game code, n is the size of the thing your algorithm scales with.

- Number of entities
- Number of tiles
- Number of jobs
- Number of resources
- Number of visible sprites
- Number of pathfinding nodes

SAE INSTITUTE

Common complexity classes

The exact cost depends on the machine and data layout, but these shapes are the first thing to recognize.

Complexity	Shape	Example
O(1)	constant	index through `std::span`
O(log n)	divide/search tree	`std::map::find`
O(n)	scan once	`std::ranges::find_if`
O(n log n)	sort/divide	`std::ranges::sort`
$O(n^2)$	nested scan	compare every entity with every entity

O(1): direct access

Direct indexing is constant time when the index is already known.

```
std::vector< Tile > tiles(width * height);
std::span< Tile > tile_view = tiles;

Tile& tile_at(std::size_t x, std::size_t y) {
    return tile_view[y * width + x];
}
```

O(log n): tree lookup

Iterating through a `std::map` is O(n). Lookup operations such as `find`, `lower_bound`, insert, and erase are O(log n).

```
std::map< ResourceId, ResourceInfo > resources;

auto it = resources.find(id);   // O(log n)

if (it != resources.end()) {
    use(it->second);
}
```

O(n): linear search

A scan over a vector is linear. It is often good enough, especially when the data is small or contiguous.

```
auto it = std::ranges::find_if(colonists,
    [](const Colonist& colonist) {
        return colonist.state == ColonistState::Hungry;
    });
```

O(n log n): sorting

Sorting is more expensive than one scan, but it can be the right setup step when many later queries become simpler.

```
std::ranges::sort(jobs, std::greater{}, &Job::priority);
```

$O(n^2)$: nested loops

Nested scans become expensive very quickly. Pairwise entity checks are the classic warning sign.

```
for (const Entity& a : entities) {
    for (const Entity& b : entities) {
        if (a.id ≠ b.id && close_enough(a, b)) {
            handle_pair(a, b);
        }
    }
}
```

Same Big-O, smaller constant

Checking each pair once avoids duplicate work, but the algorithm is still $O(n^2)$.

```
for (std::size_t i = 0; i < entities.size(); ++i) {
    const Entity& a = entities[i];

    for (std::size_t j = i+1; j < entities.size(); ++j) {
        const Entity& b = entities[j];
        if (close_enough(a, b)) {
            handle_pair(a, b);
        }
    }
}
```

Better shape: reduce the search space

Before micro-optimizing a nested loop, change the amount of work.

- Check nearby grid cells, not every entity
- Keep entities grouped by chunk
- Separate active and inactive objects
- Cache results that do not change every frame
- Use a better data structure before micro-optimizing

SAE INSTITUTE

Big-O is not the whole story

Big-O predicts growth, but it does not replace profiling.

- `std::vector` O(n) can beat `std::map` O(log n) for small data
- Contiguous memory can beat pointer chasing
- Constants matter
- Allocation matters
- Branches and cache misses matter
- Measure the real workload

What problem are you trying to solve?

Be precise before opening a profiler.

- Target frame rate: the game should run at 60 FPS, but runs at 20 FPS
- Memory budget: the mobile build uses too much memory
- Loading time: a scene takes too long to open
- Latency: an input or simulation reaction is too slow

How often does it happen?

The frequency changes the investigation.

- **Every frame**: likely in the main update/render hot path
- **Only in some states**: for example, when the camera zooms out
- **Spikes**: for example, when the player builds something or loads an area
- **Long sessions**: memory growth, fragmentation, cache churn, or accumulated state

Profilers

Profiler usage

A profiler shows where time goes, but not whether that work is necessary.

- Identify hotspots and bottlenecks
- Visualize execution time
- Collect and compute metrics
- Look for patterns over multiple samples
- Compare before/after changes

SAE INSTITUTE

Sampling profilers

Sampling profilers attach to a running program and periodically record the call stack.

- Usually works without changing the code
- Good for broad hotspot discovery
- Results are statistical averages
- Can be affected by inlining and optimization
- Examples: Intel VTune, Linux `perf`, Superluminal, platform CPU profilers

Instrumentation profilers

Instrumentation profilers record explicit zones that you add to the code.

- Zones can match business logic: `UpdateAI`, `Pathfind`, `RenderTerrain`
- You can attach custom data to the profile
- Excellent for frame-based games and simulations
- Requires code hooks, recompilation, and often a third-party dependency
- Examples: Tracy, Optick

Tracy example

Build with Tracy enabled, then attach the Tracy GUI to inspect frame time, zones, and spikes.

```
#include < tracy/Tracy.hpp >

void update_colonists(std::span< Colonist > colonists) {
    ZoneScoped;

    for (Colonist& colonist : colonists) {
#ifdef TRACY_ENABLE
        ZoneNamedN(colonistThink, "Colonist::Think", true);
#endif
        colonist.think();
    }
}

void game_frame() {
    ZoneScoped;

    update_simulation();
    render_world();

    FrameMark;
}
```

Sampling vs instrumentation

Use both when possible: sampling to discover, instrumentation to track and explain.

Sampling	Instrumentation
Attach to the program	Add zones in code
Good first pass	Good for known systems
Statistical	Exact instrumented regions
Low setup cost	More control and context
Less tied to gameplay concepts	Can match gameplay concepts

SEI INSTITUTE

Profiling workflow

Profiling loop

Never stop at “this function appears in the profiler.” Ask why it appears there.

1. Find a bottleneck
2. Decide whether it is relevant
3. Find the root cause
4. Confirm the root cause with measurement
5. Fix it
6. Measure again

Is the bottleneck relevant?

Do not optimize noise.

- Is it on the hot path?
- Is it called every frame or thousands of times per frame?
- Is it a spike the player can feel?
- Is it inside the current platform budget?
- Would fixing it move the frame time meaningfully?

Result interpretation

Once you know the bottleneck, identify the kind of time you are seeing.

- **High CPU time**: the CPU is actively doing work
- **High wait time**: the thread is idle, blocked, or waiting for another resource
- **High allocation time**: the allocator is doing work or synchronizing
- **High cache miss rate**: the CPU waits for memory instead of executing useful instructions

High CPU time

Do cache and branch work last. Algorithm and data-structure wins are usually larger.

- Inefficient algorithms: loops, recursion, repeated work
- Inefficient data structures for the access pattern
- Single-threaded work that could be parallelized
- Branch misprediction or cache misses

High wait time

Move blocking work away from the hot path, or redesign the ownership/threading model.

- Disk I/O
- Network calls
- Locks and synchronization
- Waiting for jobs or GPU work
- Thread scheduling

City-builder optimization examples

These changes only make sense after the profiler proves the cost.

- Pathfinder: replace maps with arrays where the grid bounds are known
- Pathfinder: use a linear allocator for short-lived temporary data
- Direction lookup: replace a map lookup with an array lookup
- Behavior tree update: parallelize independent AI work
- Tilemap update: split independent loops and avoid repeated range scans
- Zone manager: separate residential and commercial collections
- ~~Car/agent manager: reduce lock contention~~

Real-time hot paths

What does real-time mean?

Some game systems must answer before a deadline.

- “Fast” is not precise enough
- “Efficient” is not precise enough
- “Under 16 ms” may be true for a frame, but each subsystem has a smaller budget
- A real-time path needs predictable worst-case behavior

Real-time safe code

The code should not fail in the hot path. Do not call third-party code there unless you know what it does.

- Worst-case execution time is deterministic
- Worst-case execution time is known in advance
- Worst-case execution time is independent of unbounded application data
- Worst-case execution time is shorter than the deadline

Avoid blocking the hot path

Do not do these in real-time critical code.

- Acquire a contended mutex
- Allocate or deallocate memory
- Perform disk or network I/O
- Interact with the OS scheduler
- Make other unpredictable system calls
- Trigger hidden work through a library abstraction

Data structures

Know your data structure

Complexity tells you the shape of the work. The container decides the memory behavior and constant costs.

- Are lookups random or sequential?
- Is the size fixed or bounded?
- Do you insert/remove often?
- Do you care about stable pointers?
- Are you iterating every frame?

SAE INSTITUTE

`std::array`

The go-to container when the maximum size is known. Even with a dynamic logical size, a fixed-capacity array can be the right tool.

- Contiguous storage
- No heap allocation
- Size known at compile time
- Great cache locality
- Useful for bounded gameplay data

`std::vector`

The default dynamic sequence container. It is usually the right first dynamic container for game data.

- Contiguous heap storage
- Stores begin/end/capacity information
- Great iteration performance
- Can reallocate when growing
- Use `reserve` when you know the expected size

Queues and stacks

Often, a `std::vector` is enough.

- A vector already works as a stack with `push_back` and `pop_back`
- A queue can be implemented with a fixed-size ring buffer
- Avoid erasing from the front of a vector in a hot path
- Prefer simple contiguous storage when iteration matters

`std::map`

`std::map` is usually implemented as a red-black tree. Do not use it by habit for hot-path lookup.

- Nodes are individually allocated
- Traversal follows pointers
- Cache locality is poor
- Insertions and removals rebalance the tree
- Great when you need ordering, stable references, and logarithmic operations

Do less work

This common pattern may hash or search for the same key several times.

```
if (objects.contains(id)) {
    return objects[id].get();
}

objects[id] = std::make_unique<object>(id);
return objects[id].get();
```

Do the lookup once

The exact code is not always best. The point is to avoid repeated hidden work in a hot path.

```
auto [it, inserted] = objects.try_emplace(id, nullptr);

if (inserted) {
    it->second = std::make_unique<object>(id);
}

return it->second.get();
```

`std::unordered_map`

An unordered map is not automatically the solution. It depends on the key type, data size, access pattern, and implementation.

- Lookup requires hashing the key
- Buckets can contain chains or groups of entries
- Collisions require equality comparisons
- Rehashing can be expensive
- Memory layout is often less cache-friendly than a flat array

Hash map costs

Common problems:

- Cache misses
- Long delay before the first comparison
- Unnecessary comparisons from hash collisions
- Expensive hash functions
- Rehashing or allocation during growth

Alternatives to maps

Measure against your actual workload. The right answer depends on the problem.

- `std::array< T, N >` when keys are dense and bounded
- `std::vector< T >` indexed by integer ID
- `std::vector< std::pair < Key, Value > >` with linear search for small data
- Sorted vector plus binary search for mostly-read data
- Flat maps or dense hash maps for better locality

Allocation and strings

std::string

The answer depends on the implementation and the string length.

Godbolt: https://godbolt.org/z/944MjzvsP

```
std::string a = "hello";
std::string b = "hello sae institute gva!";
```

Small string optimization

Many `std::string` implementations store short strings inside the string object itself.

- Short strings may avoid heap allocation
- Longer strings use heap storage
- Moving a small string may copy more bytes than moving a pointer
- The exact threshold is implementation-specific

Usually allocation-free

Prefer stack storage when appropriate.

- `std::array`
- `std::pair`
- `std::tuple`
- `std::optional`
- `std::variant`

Can allocate

Know when an abstraction may touch the heap.

- `std::any`
- `std::function`
- `std::vector`
- `std::deque`
- `std::list`
- long `std::string`

Be aware of allocation

Allocation is not real-time safe. Use fixed capacity, `reserve`, arenas, pools, or custom allocators when the profiler proves allocation cost matters.

- `std::vector::push_back` can allocate when capacity is exhausted
- Growing a string can allocate
- Inserting into `std::map` or `std::list` usually allocates a node
- Allocators may synchronize internally
- Many small allocations can destroy frame-time consistency

Abstraction cost

There are no zero-cost abstractions

Abstractions are only zero-cost when the generated code is the same as the lower-level version.

- Raw pointer: https://godbolt.org/z/G4oKvqzPr
- `unique_ptr` case 1: https://godbolt.org/z/drhe9z6KG
- `unique_ptr` with noexcept: https://godbolt.org/z/6gMg6x
- `unique_ptr` and move semantics: https://godbolt.org/z/dYVRbg

Do not over-engineer

Prefer the simplest implementation that fits the measured problem.

Example: converting an enum direction to a vector can be a small table, not a complex system.

- A lookup table can beat a clever map
- A small linear search can beat a hash table
- A fixed array can beat a dynamic container
- A boring algorithm can be easier to optimize and maintain

Godbolt: https://godbolt.org/z/8YEYzYvrq

We have 16 ms, but…

- Do not spend hours making a 300 ns function called once become a 30 ns function
- Optimize the code that dominates the frame
- Hot paths are the code called constantly or with large data
- Algorithms and data structures usually matter more than micro-architecture tricks
- The goal is to ship a game that runs well

References

- Mathieu Ropert, *The Basics of Profiling*, CppCon 2021: https://www.youtube.com/watch?v=h9GGhGl-tGM
- Elizabeth Baumel, *Principles of Performant Processing*, Handmade Seattle 2021: https://vimeo.com/648348292
- Chandler Carruth, *Efficiency with Algorithms, Performance with Data Structures*, CppCon 2014: https://www.youtube.com/watch?v=fHNmRkzxHWs
- Timur Doumler, *Real-time Programming with the C++ Standard Library*, CppCon 2021: https://www.youtube.com/watch?v=Tof5pRedskI
- Malte Skarupke, *You Can Do Better than std::unordered_map*, C++Now 2018: https://www.youtube.com/watch?v=M2fKMP47slQ
- Chandler Carruth, *There Are No Zero-cost Abstractions*, CppCon 2019: https://www.youtube.com/watch?v=rHIkrotSwcc

Conclusion

- Profile with **time and memory**, not FPS.
- A profiler finds where time goes; you still need to understand whether that work is necessary.
- Use sampling profilers for broad discovery and instrumentation profilers for frame/gameplay context.
- Separate CPU work from wait time before choosing a fix.
- Real-time hot paths should avoid blocking, allocation, I/O, and unpredictable system calls.
- Choose containers based on access patterns, locality, lifetime, and allocation behavior.
- Optimize the hot path, measure again, and prefer simple code that solves the measured problem.